CS3070 – Exploration 4 - Memory Management
Summer 2025

SCORE: 0.0/75

Questions

1. Read up on the organization and operation of paged virtual memory.
    a. Make sure you can define these terms: virtual address, real address, page, frame, page table, memory mapping unit, translation lookaside buffer, demand loading, page fault, page fault handler, virtual memory swap file, multiprogramming, fixed partition, variable partition, replacement policy, eviction policy. Just attest that you have understood these terms; do not write out all the definitions.

    I attest that I understand all of these terms.

    Virtual address - generated by the CPU to designate a specific data byte in a contiguous sequence of bytes called Address Space. Example: A CPU with 16- bit addresses can access any item in the contiguous range 0, 1, 2, …, 216-1.

    Real address - designates specific location of a byte in RAM, also numbered in a contiguous sequence. Example: A RAM with 32-bit addresses can access any location in the contiguous range 0, 1, 2, …, 232-1.

    Page - unit of storage and transfer

    Frame – A fixed-size block of physical memory (RAM) into which a page from virtual memory can be loaded

    Page Table – A data structure maintained by the operating system that records the mapping between virtual pages in a process's address space and the physical frames in RAM.

    Memory Mapping Unit – Converts the linear addresses into page-line pairs, finds the frame holding the page, and generates the physical address within that frame.

    Translation lookaside buffer - Bypasses page table lookups for recent page-frame pairs.

    Demand loading - A virtual memory technique where a page is brought into RAM only when the process actually references it on demand.

    Page fault - An anomaly that occurs when a process tries to acess a virtual page that is not currently in RAM.

    Page fault handler - An interrupt in the operating system that runs when the hardware signals a page fault interrupt.

Virtual memory swap file – The copy of a process's entire address space kept on disk.

Multiprogramming - A technique where multiple processes are kept in memory at the same time so that the CPU can switch among them.

Fixed partition - A technique where each process gets a fixed amount of memory.

Variable partition - Each process gets a space that varies over time.

Replacement policy - A policy that decides which page to evict from main memory.

Eviction policy – A policy that decides which page in RAM to remove when a new page must be brought in and no free frame is available.

b. Why is the virtual memory master file needed? (Also called swap file.)

- RAM is volatile and limited in size, while processes need persistent storage for their entire address space
- It serves as the backing store for pages not currently in RAM, enabling address spaces larger than physical memory
- When page faults occur, the OS retrieves missing pages from this master file
- It ensures data consistency by maintaining authoritative copies as processes modify pages
- It automates memory management that programmers previously had to handle manually with overlays

c. What is logical partitioning and how does virtual memory achieve it?

Logical partitioning is the OS enforcing separation of address spaces, giving each process private, non-overlapping memory that appears as dedicated RAM.

Virtual memory achieves this through:

- Address spaces: Each process gets a unique virtual address space
- Page tables: Map virtual addresses to physical frames, ensuring different processes use different RAM frames
- MMU: Hardware that translates virtual addresses to physical addresses using the page tables
- Access control: Presence bits and access flags prevent unauthorized memory access between processes

2. A performance measure of virtual memory is its fault function ***F(m)***, which is the number of page faults in a measurement interval of length ***N*** references when a process allocated exactly ***m*** frames of RAM. Over the measurement interval, the *hit rate* is ***h=(N-F(m))/N***. Note that **1-*h*** is the *miss rate* or fault rate of the process. Justify this formula for the mean access time ***T***:

   T=A+(1-h)B

   where ***A*** is the time for one RAM access and ***B*** is the time for one disk access.

   We separate the above equations into two parts, the time for a single RAM access and the time for a single disk access. The reason why we multiply the miss rate with the time for disk access is because only a miss or page fault will require us to access disk memory. The mean access time required for memory access will then be the sum of both the time it takes to access RAM or (given a miss or page fault) the time it takes to access disk memory. Both of these variables are the only two locations where memory can be held (outside of the CPU).

3. (Continuing) In the following questions, suppose the RAM access time is 10 nanoseconds (10^{-8} sec) and the disk access time is 10 milliseconds (10^{-2} sec). Please **express numbers in scientific notation** with seconds as the time unit and one numeral after the decimal (e.g., 3.1×10^4).

   a. What is the mean memory access time if miss rate is 3%?

      $A = 1.0 \times 10^{-8}$ s
      $B = 1.0 \times 10^{-2}$ s
      $T = A + (1-h)B$
      $T = (1.0 \times 10^{-8}\text{ s}) + (0.03)(1.0 \times 10^{-2}\text{ s}) = 1.0 \times 10^{-8} + 3.0 \times 10^{-4} = 3.0 \times 10^{-4}$ s

   b. What miss rate would be required to get the mean memory access time to within 2% of the basic RAM time?

      $T \leq (1.02)(1.0 \times 10^{-8}\text{ s}) = 1.02 \times 10^{-8}$ s

      $1.02 \times 10^{-8} \geq (1.0 \times 10^{-8}) + (\text{miss_rate})(1.0 \times 10^{-2})$

      $\text{miss_rate} \leq (1.02 \times 10^{-8} - 1.0 \times 10^{-8})/(1.0 \times 10^{-2})$

      $\text{miss_rate} \leq (2.0 \times 10^{-10})/(1.0 \times 10^{-2}) = 2.0 \times 10^{-8}$

   c. The memory efficiency ***E*** is the fraction of time a process is using RAM (and not DISK). Justify this formula:

      E=A/T

      This is a simple ratio, with A being the amount of time that RAM is accessed compared to total time that memory is being accessed. T will

give us the time that either RAM or disk storage is accessed, so E is the ratio of only when RAM is accessed.

d. What is the memory efficiency of a process of miss rate 1%?

Let's use the above example in (a) but adjusting for a miss rate of 1%.

$A = 1.0 \times 10^{-8}$ s

$B = 1.0 \times 10^{-2}$ s

$T = A + (1-h)B$

$T = (1.0 \times 10^{-8}\ s) + (0.01)(1.0 \times 10^{-2}\ s) = 1.0 \times 10^{-8} + 1.0 \times 10^{-4} = 1.0 \times 10^{-4}$ s

$E = A/T = (1.0 \times 10^{-8}\ s)/(1.0 \times 10^{-4}\ s) = 1.0 \times 10^{-4}$

This means that with a 1% miss rate, RAM is accessed only 0.01% of the time while waiting on disk the other 99.99% of the time.

e. Estimate how many processes of (d) would need to be active together to achieve RAM efficiency of 100%.

If each process uses RAM E fraction of the time, then the number of processes needed is: $N = 1/E$

$N = 1/(1.0 \times 10^{-4}) = 1.0 \times 10^{4}$

This equals 10,000 processes required with a 1% miss rate to keep the RAM 100% busy.

4. A **page replacement algorithm** is a component of virtual memory that decides which page of a process to evict when the process has a page fault and its memory allocation of *m* pages is full. We are interested in the fault function ***F(m)*** for different paging algorithms, starting with initially empty RAM.

a. The notes Ch 4.4 of Module 4 contain worked simulation examples of memory contents for a reference string under FIFO, LRU, and MIN. Review all these examples with your team and be sure that you know how to construct these memory charts on your own. Just attest that you have done this; no need to write it all out.

I attest that I have reviewed FIFO, LRU, and MIN.

FIFO - first in first out

• m loaded pages in a circular list

• Index i tells which page is next for replacement

• On page fault, replace that page and set i=(i+1) mod m

• Pro: very simple implementation

• Con: consistently more faults than LRU

LRU - least recently used

• Maintain vector called "LRU stack" ordering all M pages from most to least recently referenced

• Page to be replaced is at position m in the stack • Pro: good performance, no anomalies

• Con: high overhead of constantly updating the stack

MIN - minimum possible faults (optimum)

• Replace page with longest forward distance until next reuse

• Attraction: benchmark for realizable policies

• Con: not realizable

• Can be used as post mortem benchmark to compare with LRU and FIFO – usually significantly better than LRU

b. In the simulations of FIFO you will notice that for this page reference string the number of page faults increases when available memory is increased from 3 to 4 pages. This is called *Belady's Anomaly*. Why is it called an anomaly? Explain the anomaly and its root cause.

This FIFO simulation is called an anomaly because we would normally expect that as available memory increases, the number of page faults decreases or stays the same. It is commonly assumed that as the number of frames are available, you receive less page faults.

In *Belady's Anomaly,* the number of page faults with 3 frames is 9, but with 4 frames is 10. This is the opposite outcome of what you would expect.

The root cause of this anomaly occurs because FIFO does not satisfy the inclusion property. With FIFO, adding additional frames changes the first in first out rotation. This means that FIFO could potentially throw away pages which actually worked with smaller memory. In this case, the memory contents for 3 are not subsets for those in 4.

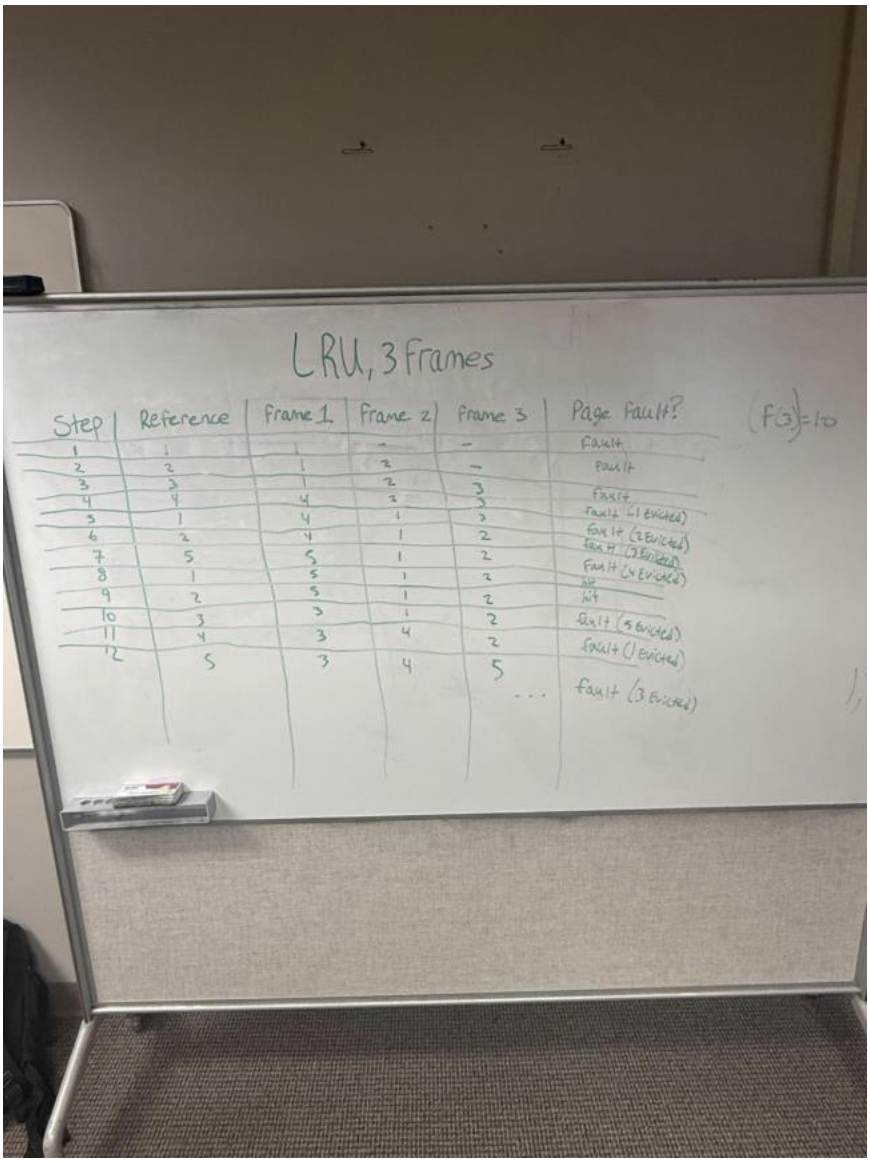
c. In the simulations of LRU, you will notice that the memory contents when ***m*** pages are allocated are always a subset of the contents when ***m+1*** pages are allocated. This is called the *inclusion property*. Why does LRU always satisfy the inclusion property?

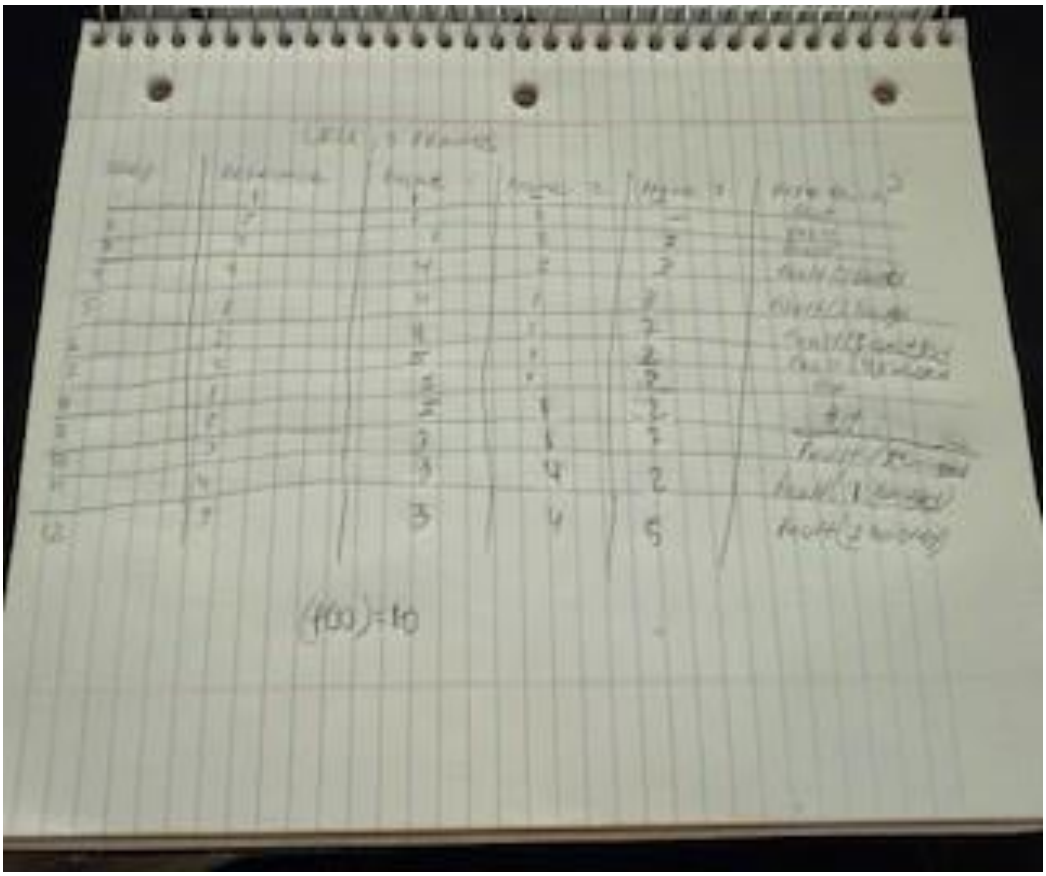
LRU always keeps the m most recently used pages in memory. When memory increases to m+1 frames, LRU simply extends to keep the m+1 most recently used pages. Since the m most recent pages are always a subset of the m+1 most recent pages, the inclusion property is satisfied - no previously kept pages are discarded.

d. Is LRU immune to Belady's Anomaly? Explain.

Yes, LRU is immune to Belady's Anomaly because it satisfies the inclusion property. Since all pages present in m frames will remain present when memory increases to m+1 frames, the number of page faults can never increase - only decrease or stay the same.

e. Any paging algorithm that satisfies the **inclusion property** can represent its memory contents as a **stack**, which is a list of the pages such that the first *m*-entries of the stack are the contents of *m*-page memory. We can construct the LRU stack in real time as we scan new references by a simple stack update procedure. The position of the referenced page in the previous stack is called its **stack distance**. Show that ***F(m)*** for LRU is the number of page faults. Stack distance is an important concept in page replacement analytics. (The first 20 pages of Ch 4.5 of Module 4 will be helpful to you.)





Stack distance = d(t)

Position of the referenced page = r(t)

Previous stack = S(t-1)

First references are treated as d(t) = infinity

Page fault if and only if d(t) > m

If d(t) ≤ m, the reference page is already in the first m entries of S(t-s) → no fault

If d(t) > m, then it was not in the m page memory → fault. The page must be brought in.

c(k) = histogram. Count how many times d(t) = k across the whole trace, for k = 1,...,M, and let c(M + 1) count first references

F(m) = |{t : d(t) > m}| = c(k) *** add M=1 and k=m+1

S(t) = the stack at time t

Ordered list of all pages of the program at that moment

Stack arranged from most to least recently used

Current reference r(t) = i always placed at the top of S(t)

The first m entries of S(t) represents the contents of an m page memory under LRU at time t.

So S(t-1) is the ordering just before reference r(t)

D(t), the stack distance, is the position of r(t) in S(t-1)

Updating S(t-1) to S(t) means moving the referenced page to the top and pushing down the intervening pages.

*** NEW IDEA TO DESCRIBE ***

Write the reference string across the top.

Draw m rows (for the m frames in RAM)

Step through each reference left to right

If the page is already in RAM then hit

If not then fault then load it. If RAM is full, apply the eviction policy and replace the victim

Mark each page fault with an X

Count them at the end = F(m)

f. In the simulations of MIN, you will notice that MIN satisfies the inclusion property and that the updates to properly order the stack pages are more complex than LRU because they depend on the future order of pages. Yet ***F(m)*** for MIN is also the number of stack distances > *m*. Why is this?

MIN is a stack algorithm where the stack is ordered by increasing forward distance to next reference. Like LRU, it satisfies the inclusion property - the first m entries represent the contents of m-page memory. The fault condition is identical: a fault occurs when d(t) > m (the referenced page is below the m-line in the stack). Therefore, F(m) equals the count of stack distances > m.

g. Why does no operating system use MIN?

MIN requires perfect knowledge of future memory references to select the page with the longest forward distance for replacement. Since operating systems cannot predict future program behavior, MIN is not realizable and serves only as a theoretical benchmark for evaluating practical algorithms.

5. Multiprogramming gives us a new degree of freedom that enables more efficient use of memory: we are not constrained to keep a process's memory allocation fixed. Evictions need not coincide with faults. A process's memory allocation can grow when the process needs more and shrink when it needs less. **Working set** (WS) is a well-known paging algorithm for variable space allocation. It uses a **window** of size *T* units to look backward at the *T* most recent page references (including the current reference). See the definitions of WS in Ch 4.6 of Module 4. Notice that with working set, memory is dynamically partitioned, and each process gets pages sufficient to hold its working set. There are some **free pages** not allocated to any working set. The job scheduler adds a new process to memory only when **free** is large enough to hold its working set.

    a. Why can the working sets after each reference be different in size?

       Working sets can be different in size after each reference because the T-window slides with each new reference, potentially adding new pages while dropping pages that fall outside the window.

    b. When are new pages added to a working set? Where do they come from?

       New pages are added on page faults when a referenced page isn't in the working set. They come from free memory, or if free memory is empty, from evicting the LRU page within that same working set.

    c. When are pages evicted from a working set? Where do they go?

       Pages are evicted when they haven't been used within the window size T (not just falling outside the reference window). They go to free memory.

    d. Why are additions and evictions not necessarily synchronized?

       Additions and evictions are not necessarily synchronized because they're driven by different events: additions occur on page faults/new references, while evictions occur when pages age out of the time window.

    e. Thrashing can happen in a memory that is partitioned among multiple processes using **global LRU**: at a page fault, the page not used for the longest time among all pages in memory is evicted from memory. Explain how thrashing can happen under the global LRU policy.

       Process A faults and evicts Process B's page. Process B faults and evicts Process C's page. Process C faults and evicts Process A's page. This cycle repeats endlessly as each process keeps losing the pages it needs to the global LRU policy.

    f. What is happening in the system when it is thrashing? Is thrashing a form of deadlock?

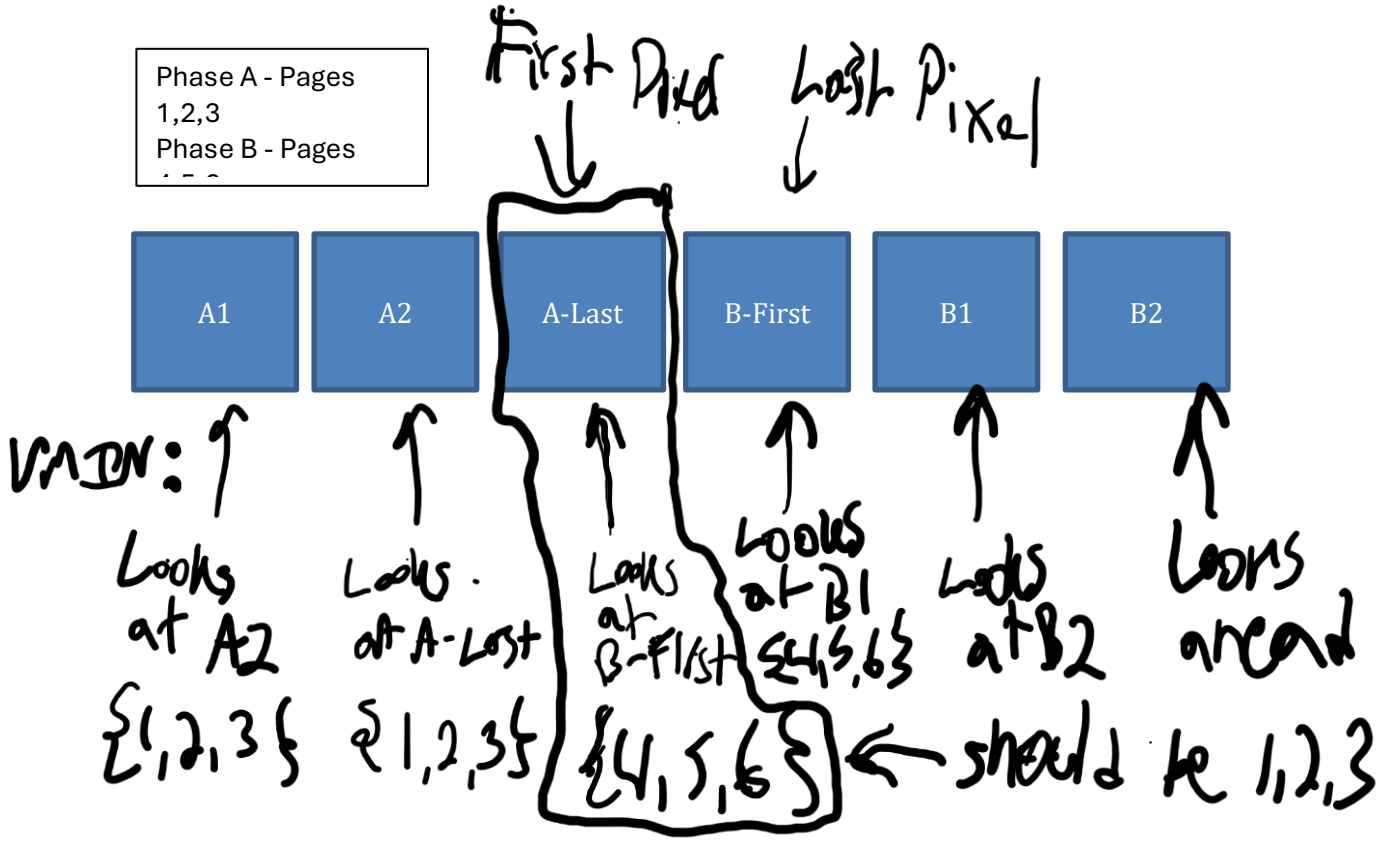
The CPU becomes largely idle as processes waiti in the I/O queus for page loads. Thrashing is like starvation, not deadlock, as processes wait on I/O due to insufficient memory rather than forming cyclic resource dependencies.

g. Why is WS immune to thrashing?

   WS algorithm prevents page stealing between processes by ensuring each gets sufficient pages for its working set, and the job scheduler only admits new processes when enough free memory exists for their working set.

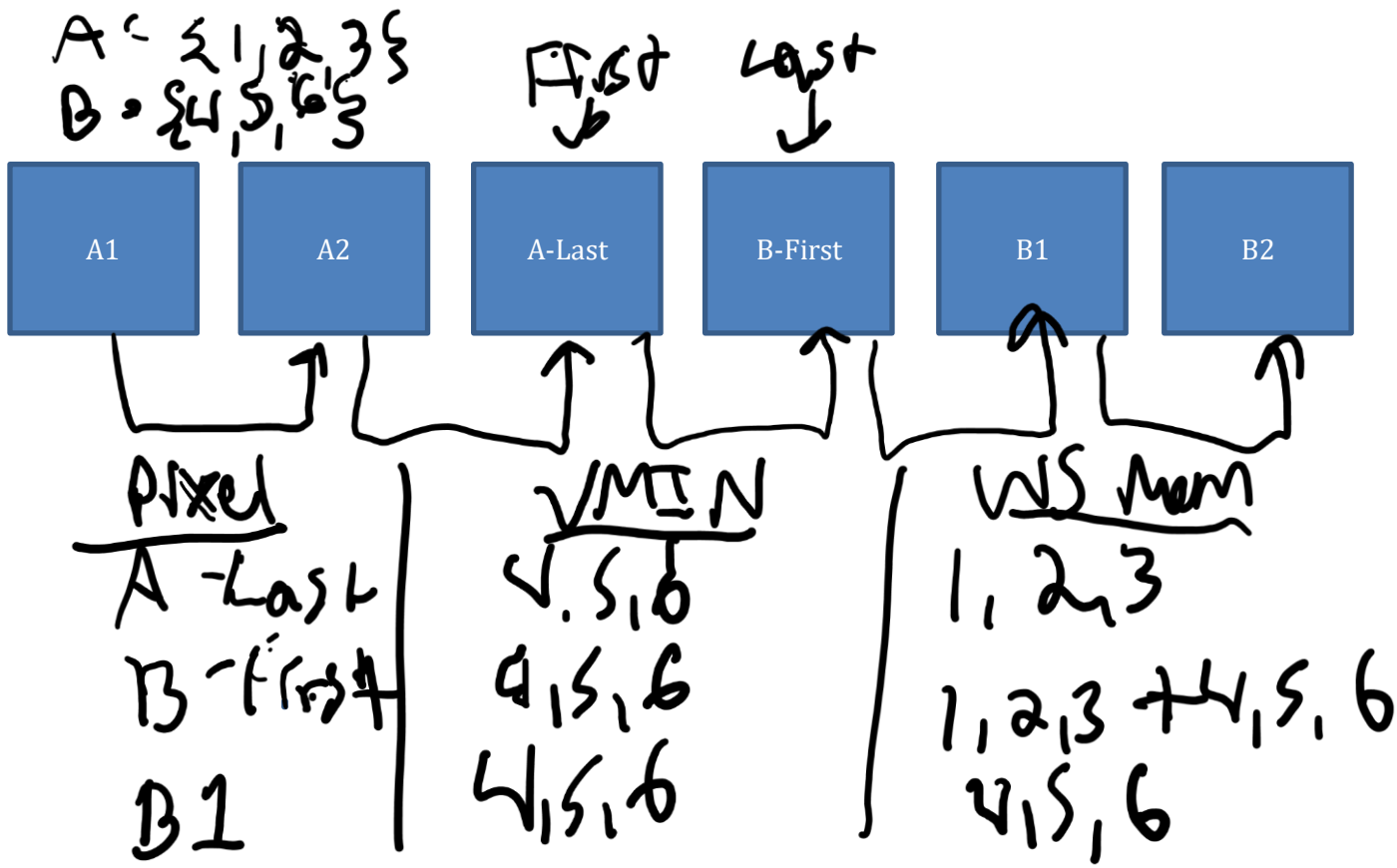
6. In Ch 4.6 of Module 4 the VMIN policy is considered. VMIN is the counterpart of MIN for variable space. It is the best possible algorithm compared to all other fixed and variable space algorithms. Like WS, it uses a window to decide which pages are important. Unlike working set, its window is a *lookahead* into the next T references of the future. VMIN is often significantly better than MIN because MIN is constrained to a fixed size memory.

   a. Ch 4.6 show a page reference map from a Firebox browser. Review the definitions of page reference map and how it is measured. The map clearly shows the program using different page sets ("locality sets") during different time phases. There are five main phases in the figure. Suppose VMIN could be applied to the Firefox browser with its window size equal to one pixel (approximately 376K references). Show that VMIN memory contents equals the locality set for all pixels in phase except the first and the last.



With window size of one pixel, VMIN memory = locality set for middle pixels only. At transitions (first/last), VMIN holds the pages for the next locality set, not the current one.

b. Now show that, with window size one pixel, WS contents are identical to the VMIN contents except for the first and last pixel of a phase. For a phase of ***L*** pixels, give a formula for the fraction of time during WS and VMIN are equal. What is the fraction for a phase with ***L=200***? ***L=50***? (Those are the longest and shortest phases in the Firefox diagram.). Does this justify the claim that WS and VMIN are nearly identical for programs with locality?



VMIN and WS are identical in middle pixels. They differ at transitions because VMIN anticipates the future while WS has overhang from the past.

Formula for fraction of time WS and VMIN are equal for phase of L pixels: (L-2)/L.

When L=200, (200-2)/200 = 198/200 = 99%

When L=50, (50-2)/50 = 48/50 = 96%

This justifies the claim that WS and VMIN are nearly identical for programs with locality as the above calculations show an observed 50-200 pixels have 96-99% overlap. This difference in percentage is attributed to WS overhang. The close percentage shows the principle of locality.